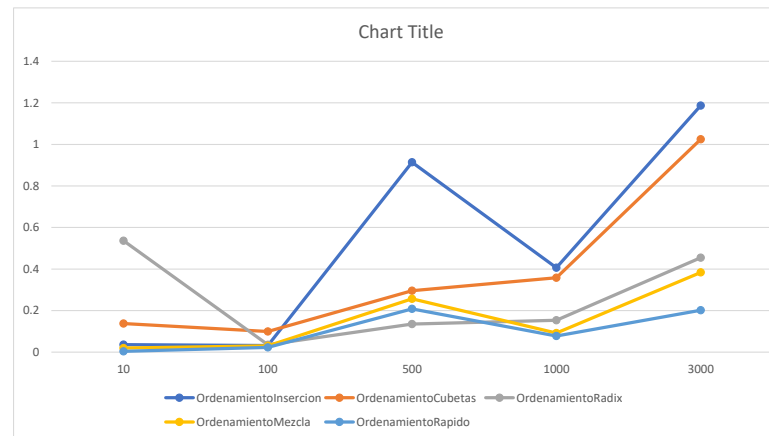


Cantidad	Tiempo	Algoritmo
10	0,036	OrdenamientoInsercion
10	0,0198	OrdenamientoMezcla
10	0,004	OrdenamientoRapido
10	0,5364	OrdenamientoRadix
10	0,1379	OrdenamientoCubetas
100	0,0318	OrdenamientoInsercion
100	0,0307	OrdenamientoMezcla
100	0,0227	OrdenamientoRapido
100	0,0357	OrdenamientoRadix
100	0,0994	OrdenamientoCubetas
500	0,9137	OrdenamientoInsercion
500	0,2569	OrdenamientoMezcla
500	0,2081	OrdenamientoRapido
500	0,1354	OrdenamientoRadix
500	0,2955	OrdenamientoCubetas
1000	0,4061	OrdenamientoInsercion
1000	0,092	OrdenamientoMezcla
1000	0,0775	OrdenamientoRapido
1000	0,1537	OrdenamientoRadix
1000	0,3578	OrdenamientoCubetas
3000	1,1871	OrdenamientoInsercion
3000	0,3835	OrdenamientoMezcla
3000	0,2018	OrdenamientoRapido
3000	0,455	OrdenamientoRadix
3000	1,0248	OrdenamientoCubetas

Suma de Tiempo	Algoritmo					
Cantidad	OrdenamientoInsercion	OrdenamientoCubetas	OrdenamientoRadix	OrdenamientoMezcla	OrdenamientoRapido	Total general
10	0,036	0,1379	0,5364	0,0198	0,004	0,7341
100	0,0318	0,0994	0,0357	0,0307	0,0227	0,2203
500	0,9137	0,2955	0,1354	0,2569	0,2081	1,8096
1000	0,4061	0,3578	0,1537	0,092	0,0775	1,0871
3000	1,1871	1,0248	0,455	0,3835	0,2018	3,2522
Total general	2,5747	1,9154	1,3162	0,7829	0,5141	7,1033



El análisis de los algoritmos muestra cómo se comportan con los diferentes tamaños de entrada (10, 100, 500, 1000 y 3000). El Ordenamiento por Inserción es eficiente con pocos elementos, pero crece cuando se hace más grande la lista. El Ordenamiento de Mezcla y el Rápido mantienen tiempos bajos en todos los tamaños de entrada, lo que nos dice que son una buena opción para grandes volúmenes de datos. Los algoritmos Radix y Cubetas muestran desempeño irregular. Los tiempos totales nos dicen que el Ordenamiento Rápido es el más eficiente e Inserción es el más ineficiente, pero no lo hace inútil sino más eficiente para listas pequeñas, mientras que para listas grandes son más útiles los algoritmos de Mezcla y Rápido.